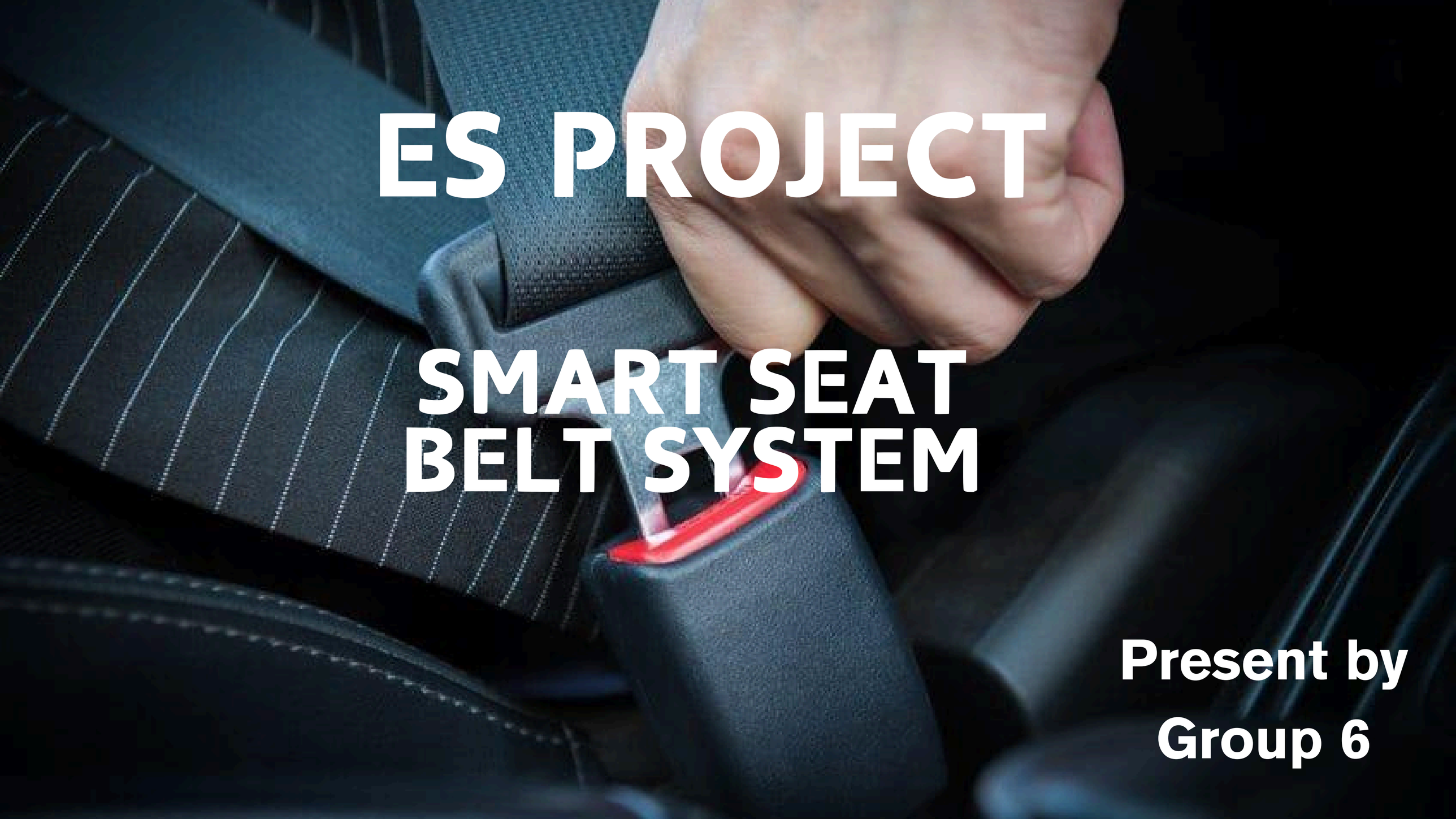


ES PROJECT

SMART SEAT BELT SYSTEM

Present by Group 6

OUR TEAM

Pranav S

N Bhanu Prakash

GR Yogananth

M Uday Chaitanya

S Harsha Vardhan Raju

INTRODUCTION

Our project introduces a Smart Seat Belt System aimed at protecting passengers during accidents by automatically tightening the seat belt to reduce the risk of injuries. Using sensors like ultrasonic, vibration detectors and Stepper motor, the system monitors surroundings in real-time and tightens the seat belt during danger. It also sends instant alerts via Bluetooth in accidents, addressing safety issues like loose belts and delayed emergency responses.

-:COMPONENTS:-

01 Arduino Uno

02 Ultrasonic Sensor

03 Bluetooth Module

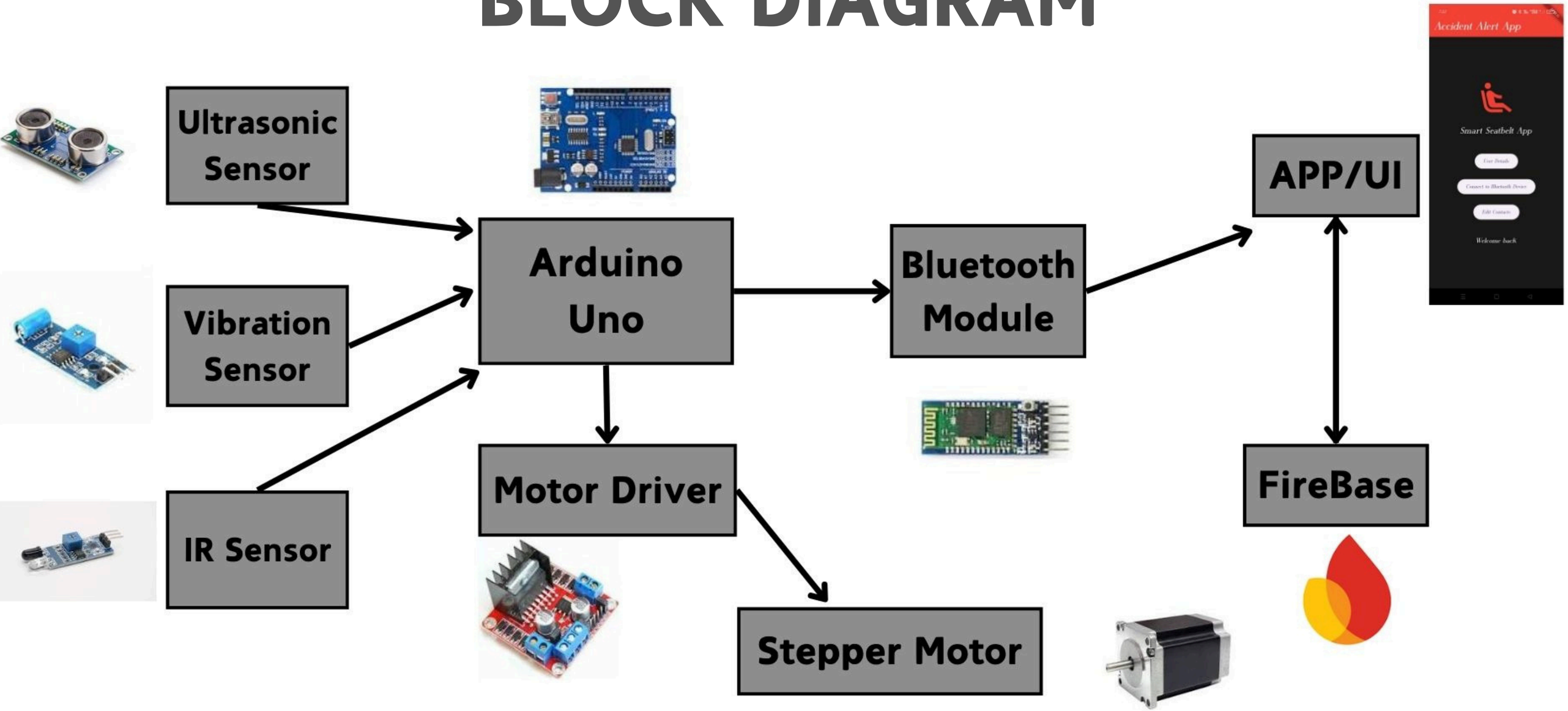
04 Stepper Motor

05 Motor Driver

06 IR Sensor

07 Vibration Sensor

BLOCK DIAGRAM



MAIN COMPONENTS

Vibration Sensor

To detect vibration, if vibration is detected and if object is under certain distance threshold then we assumed it as an accident.

IR Sensor

To adjust seat belt tightening we are using an IR sensor and white sheet.

Ultrasonic Sensor

It is used to check whether a vehicle is in accidental zone of our car.

Bluetooth Module

If the vehicle is in accidental zone and vibration is detected then the bluetooth module sends a string to the app indicating that the vehicle in an accident.

SCREENSHOTS OF CODES

bluetooth_ultrasonicsensor_vibrationsensor_steppermotor | Arduino IDE 2.3.3

File Edit Sketch Tools Help

Arduino Uno

bluetooth_ultrasonicsensor_vibrationsensor_steppermotor.ino

```
#define A11 8
#define A12 9
#define B21 10
#define B22 11
#define trigger 2

#define VIBRATION_SENSOR_PIN A0
#define IR_SENSOR_PIN A1


#include <SoftwareSerial.h>
SoftwareSerial bluetooth(12, 13);


int steps[4][4] = {
    {LOW, HIGH, HIGH, LOW},
    {LOW, HIGH, LOW, HIGH},
    {HIGH, LOW, LOW, HIGH},
    {HIGH, LOW, HIGH, LOW}
};


const float distanceThreshold = 5.0;
const int validReadingsRequired = 1;
const int irThreshold = 500;
void setup() {

    pinMode(A11, OUTPUT);
    pinMode(A12, OUTPUT);
    pinMode(B21, OUTPUT);
    pinMode(B22, OUTPUT);
    pinMode(trigger, OUTPUT);
    for (int i = 3; i <= 6; i++) {
        pinMode(i, INPUT);
    }
    pinMode(VIBRATION_SENSOR_PIN, INPUT);
    pinMode(IR_SENSOR_PIN, INPUT);


    Serial.begin(9600);
    bluetooth.begin(9600);
    Serial.println("System Initialized...");
}

void rotateMotorForTenSeconds() {
    Serial.println("Starting motor rotation...");
```

Output Serial Monitor

```
Sketch uses 5008 bytes (15%) of program storage space. Maximum is 32256 bytes.
Global variables use 629 bytes (30%) of dynamic memory, leaving 1419 bytes for local varia
```

bluetooth_ultrasonicsensor_vibrationsensor_steppermotor | Arduino IDE 2.3.3

File Edit Sketch Tools Help

Arduino Uno

bluetooth_ultrasonicsensor_vibrationsensor_steppermotor.ino

```
    while (analogRead(IR_SENSOR_PIN) > irThreshold) {
        for (int i = 0; i < 4; i++) {
            digitalWrite(A11, steps[i][0]);
            digitalWrite(A12, steps[i][1]);
            digitalWrite(B21, steps[i][2]);
            digitalWrite(B22, steps[i][3]);
            delay(5);
        }
    }
}


void holdMotorTorqueForTenSeconds() {
    unsigned long startTime = millis();

    while (millis() - startTime < 10000)
    {

        digitalWrite(A11, HIGH);
        digitalWrite(A12, LOW);
        digitalWrite(B21, HIGH);
        digitalWrite(B22, LOW);
        delay(5);
    }
}

float ultrasonicSensor(int triggerPin, int echoPin) {
    digitalWrite(triggerPin, LOW);
    delayMicroseconds(2);
    digitalWrite(triggerPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(triggerPin, LOW);
    float duration = pulseIn(echoPin, HIGH);
    return duration * 0.0343 / 2;
}

void loop() {

    for (int i = 3; i <= 6; i++) {
        int validReadings = 0;
        for (int j = 0; j < validReadingsRequired; j++) {
            float distance = ultrasonicSensor(trigger, i);
            if (distance > 0 && distance < distanceThreshold) {
                validReadings++;
            } else {
                validReadings = 0;
```

Output Serial Monitor

```
Sketch uses 5008 bytes (15%) of program storage space. Maximum is 32256 bytes.
Global variables use 629 bytes (30%) of dynamic memory, leaving 1419 bytes for local variables.
```

```
                validReadings = 0;
                break;
            }
            delay(50);
        }
        if (validReadings == validReadingsRequired) {
            Serial.println("Object detected within threshold. Rotating motor...");
            rotateMotorForTenSeconds();
            holdMotorTorqueForTenSeconds();
            int vibrationStatus = digitalRead(VIBRATION_SENSOR_PIN);
            if (vibrationStatus == HIGH) {
                Serial.println("Vibration detected after motor rotation and holding.");

                bluetooth.println("A");
                Serial.println("Bluetooth message sent: Accident happened!");
            } else {
                Serial.println("No vibration detected after motor rotation and holding.");
            }
        } else {
            Serial.println("No object detected within threshold.");
        }
    }
    delay(50);
}
```

Output Serial Monitor

```
Sketch uses 5008 bytes (15%) of program storage space. Maximum is 32256 bytes.
Global variables use 629 bytes (30%) of dynamic memory, leaving 1419 bytes for local variables. Maximum is 2048
```

```
Future<void> sendLocationSms() async {
  try {
    print("[DEBUG] Starting sendLocationSms function.");

    // Attempting to retrieve location
    print("[DEBUG] Attempting to get location...");
    Position position = await getUserLocation();
    print("[DEBUG] Location retrieved: Latitude=${position.latitude}, Longitude=${position.longitude}"
    String locationUrl = 'https://www.google.com/maps?q=${position.latitude},${position.longitude}';
    print("[DEBUG] Generated location URL: $locationUrl");
    print("[DEBUG] Requesting SMS permissions...");
    bool? permissionsGranted = await telephony.requestSmsPermissions;
    if (permissionsGranted ?? false) {
      print("[DEBUG] SMS permissions granted.");
      print("[DEBUG] Fetching emergency contacts from Firestore...");
      List<String> contacts = await getEmergencyContacts();
      print("[DEBUG] Emergency contacts retrieved: $contacts");

      for (String contact in contacts) {
        bool smsSent = false;
        int retryCount = 0;
        while (!smsSent) {
          try {
            print("[DEBUG] Attempting to send SMS to $contact (Retry: $retryCount)...");
            await telephony.sendSms(
              to: contact,
              message: 'My current location: $locationUrl',
            );
            print("[DEBUG] SMS successfully sent to $contact.");
            smsSent = true; // Mark SMS as sent
          } catch (e) {
            retryCount++;
            print("[DEBUG] Failed to send SMS to $contact. Error: $e");
            print("[DEBUG] Retrying in 2 seconds...");
            await Future.delayed(Duration(seconds: 2)); // Wait before retrying
          }
          if (retryCount >= 5) {
            print("[DEBUG] Maximum retry limit reached for $contact. Skipping...");
            break;
```

CHALLENGES FACED

During the project, we faced several challenges. One of the initial hurdles was implementing the entire structure, but we overcame it by creating a small wooden board to support the components. Another issue was measuring both velocity and distance at the same time with the ultrasonic sensor, so we decided to only measure the distance instead. To determine how tight the seatbelt should be, we used an IR sensor. We also encountered issues with the app, where it successfully sent the location only 7 out of 10 times, leaving the remaining 3 times without any transmission. Additionally, there were delays in the belt tightening mechanism, which required further adjustments to ensure a smoother operation.

CONCLUSION

The Smart Seat Belt System enhances vehicle safety by combining advanced sensor technology and automation. It ensures passengers are securely restrained during accidents by tightening the seat belt in response to potential dangers. With features like real-time monitoring, automatic adjustments, and emergency alerts via Bluetooth, this system addresses critical safety challenges, such as improper seat belt use and delayed emergency responses. By integrating this innovation, we can significantly reduce injuries and fatalities, making roads safer for everyone.

THANK
YOU